

Governo do Estado de São Paulo
Secretaria de Desenvolvimento Econômico, Ciência e Tecnologia
Centro Estadual de Educação Tecnológica Paula Souza
ETEC

BANCO DE DADOS II

Apostila do Aluno

Curso Técnico em Desenvolvimento de Sistemas
Eixo Tecnológico: Informação e Comunicação

2026

Sumário

Capítulo 1 – Fundamentos de Bancos de Dados Não Relacionais	3
Capítulo 2 – Banco de Dados de Documentos (MongoDB)	7
Capítulo 3 – Banco de Dados Colunar (Cassandra)	12
Capítulo 4 – Banco de Dados Chave-Valor (Redis)	16
Capítulo 5 – Outros Modelos e Arquiteturas	20
Exercícios de Fixação	25
Referências Bibliográficas.....	33

Capítulo 1 – Fundamentos de Bancos de Dados Não Relacionais

1.1 Terminologia NoSQL e NewSQL

O termo NoSQL (Not Only SQL) foi popularizado no final dos anos 2000 para designar bancos de dados que não utilizam exclusivamente o modelo relacional e a linguagem SQL como principal meio de consulta. Diferentemente do que muitos pensam, NoSQL não significa "sem SQL", e sim "não apenas SQL", reconhecendo que esses sistemas podem coexistir com bancos relacionais tradicionais.

NewSQL, por sua vez, é um termo mais recente que descreve bancos de dados relacionais modernos que buscam oferecer a escalabilidade e o desempenho dos sistemas NoSQL sem abrir mão das garantias ACID (Atomicidade, Consistência, Isolamento e Durabilidade) típicas dos bancos relacionais clássicos. Exemplos incluem Google Spanner, CockroachDB e VoltDB.

1.2 Diferenças em relação a bancos de dados relacionais

Bancos relacionais (como MySQL, PostgreSQL e SQL Server) organizam os dados em tabelas com esquemas rigidamente definidos, utilizam SQL para consultas e garantem as propriedades ACID. Já os bancos NoSQL adotam modelos de dados mais flexíveis, como documentos, grafos, pares chave-valor e colunas. As principais diferenças são:

- **Modelo de dados:** Esquema flexível vs. esquema rígido
- **Escalabilidade:** Escalabilidade horizontal nativa vs. vertical
- **Desempenho:** BAIXA latência em grandes volumes
- **Consistência:** AP (disponível e tolerante a partição) no Teorema CAP
- **Linguagem de consulta:** APIs específicas vs. SQL padronizado

1.3 Motivação, aplicações, vantagens e desvantagens

A principal motivação para adoção de bancos NoSQL é a necessidade de processar grandes volumes de dados não estruturados ou semiestruturados, com alta disponibilidade e baixa latência, típicos de aplicações web modernas, redes sociais, IoT e análise de big data.

Vantagens:

- Alta escalabilidade horizontal
- Esquema flexível que acelera o desenvolvimento
- Desempenho otimizado para operações específicas
- Modelos de dados próximos ao domínio da aplicação

Desvantagens:

- Maturidade inferior aos bancos relacionais
- Ferramentas de administração menos robustas
- Ausência de suporte a joins complexos e transações ACID completas (na maioria)

- Curva de aprendizado para modelagem não relacional

1.4 Modelagem e esquema: dados estruturados, semiestruturados e não estruturados

Dados estruturados possuem esquema fixo e são armazenados em tabelas com tipos bem definidos (ex.: relacional). Dados semiestruturados não exigem esquema rígido, mas possuem metadados que descrevem sua estrutura (ex.: JSON, XML, BSON). Dados não estruturados não possuem formato predefinido (ex.: imagens, vídeos, arquivos de texto livre).

Bancos NoSQL são particularmente adequados para dados semiestruturados e não estruturados, permitindo que cada registro tenha seu próprio conjunto de campos sem necessidade de alterações no esquema global.

1.5 Sistemas distribuídos e consistência eventual

Sistemas distribuídos são compostos por múltiplos nós que se comunicam via rede para oferecer um serviço único. Nesse contexto, a consistência eventual é um modelo de consistência fraca no qual, após um período sem novas escritas, todos os nós do sistema convergem para o mesmo estado. Esse modelo é amplamente adotado por bancos NoSQL para garantir alta disponibilidade e desempenho em cenários de replicação assíncrona.

1.6 Teorema CAP (consistência, disponibilidade, tolerância de partição)

Formulado por Eric Brewer em 2000, o Teorema CAP afirma que um sistema distribuído pode garantir no máximo duas das três propriedades a seguir simultaneamente:

- Consistência (C): todos os nós enxergam os mesmos dados ao mesmo tempo.
- Disponibilidade (A): toda requisição recebe uma resposta (bem-sucedida ou não).
- Tolerância a partição (P): o sistema continua operando mesmo com falhas de comunicação entre nós.

Na prática, sistemas distribuídos precisam tolerar partições de rede (P), restando a escolha entre consistência (CP) ou disponibilidade (AP). MongoDB, por exemplo, prioriza CP; Cassandra prioriza AP.

Capítulo 2 – Banco de Dados de Documentos (MongoDB)

2.1 Conceitos: coleções, documentos, esquema flexível

MongoDB é um banco de dados NoSQL orientado a documentos que armazena dados em formato BSON (Binary JSON). Os principais conceitos incluem:

- Documento: unidade básica de dados, similar a um JSON. Exemplo:

```
{ "_id": 1, "nome": "Maria", "curso": "DS", "notas": [9.5, 8.0] }
```
- Coleção: conjunto de documentos (análogo a uma tabela no relacional).
- Esquema flexível: documentos em uma mesma coleção podem ter campos diferentes.

2.2 Operações CRUD (insert, find, update, delete)

Insert:

```
db.alunos.insertOne({ nome: "João", idade: 18, curso: "DS" })
db.alunos.insertMany([ { nome: "Ana" }, { nome: "Pedro" } ])
```

Find:

```
db.alunos.find({ idade: { $gte: 18 } })
db.alunos.findOne({ nome: "João" })
```

Update:

```
db.alunos.updateOne({ nome: "João" }, { $set: { idade: 19 } })
db.alunos.updateMany({ curso: "DS" }, { $inc: { semestre: 1 } })
```

Delete:

```
db.alunos.deleteOne({ nome: "Pedro" })
db.alunos.deleteMany({ idade: { $lt: 18 } })
```

2.3 Linguagem de consulta (filtros, projeção, operadores)

O MongoDB possui uma rica linguagem de consulta baseada em documentos. Os filtros utilizam operadores de comparação (\$eq, \$ne, \$gt, \$gte, \$lt, \$lte) e lógicos (\$and, \$or, \$not).

```
// Alunos com idade entre 18 e 25 anos
db.alunos.find({ idade: { $gte: 18, $lte: 25 } })
```

A projeção permite selecionar campos específicos no resultado, economizando largura de banda:

```
db.alunos.find({}, { nome: 1, curso: 1, _id: 0 })
```

Expressões regulares e pesquisas em arrays também são suportadas:

```
db.alunos.find({ nome: /^Mari/i })
db.alunos.find({ tags: "python" })
```

2.4 Agregação (pipeline, \$match, \$group, \$sort)

O framework de agregação do MongoDB processa documentos através de um pipeline de estágios. Cada estágio transforma os dados de entrada e os repassa para o próximo.

```
db.vendas.aggregate([
  { $match: { data: { $gte: ISODate("2025-01-01") } } },
  { $group: { _id: "$produto", total: { $sum: "$quantidade" } } },
  { $sort: { total: -1 } },
  { $limit: 10 }
])
```

Outros estágios importantes incluem \$project, \$unwind, \$lookup (para joins) e \$bucket.

2.5 Transações e modelo de consistência

A partir da versão 4.0, o MongoDB passou a suportar transações multi-documento em ambientes de replica set. A partir da versão 4.2, transações também são suportadas em sharded clusters. As transações seguem as propriedades ACID, garantindo atomicidade e isolamento.

```
const session = db.getMongo().startSession()
session.startTransaction()
try {
  db.clientes.insertOne({ _id: 1, nome: "João" }, { session })
  db.pedidos.insertOne({ clienteId: 1, total: 150 }, { session })
  session.commitTransaction()
} catch (e) {
  session.abortTransaction()
} finally {
  session.endSession()
}
```

O modelo de consistência do MongoDB é configurável: é possível ajustar o nível de preocupação de leitura (readConcern) e escrita (writeConcern) para equilibrar consistência e desempenho.

Capítulo 3 – Banco de Dados Colunar (Cassandra)

3.1 Arquitetura: anéis, replicação, partições

Apache Cassandra é um banco de dados NoSQL distribuído, baseado no modelo colunar, projetado para lidar com grandes volumes de dados em clusters distribuídos sem ponto único de falha. Sua arquitetura é inspirada no DynamoDB (Amazon) e no Bigtable (Google).

Os nós são organizados em um anel (ring), onde cada nó é responsável por um intervalo de partições (tokens). A replicação é configurada pelo fator de replicação (RF), que determina em quantos nós cada registro é copiado. O particionamento é feito com base na partition key.

O Cassandra utiliza o protocolo Gossip para descoberta de nós e o Snitch para determinar a topologia de rede, permitindo roteamento eficiente de requisições.

3.2 Modelagem orientada a consultas

Diferentemente do modelo relacional (que normaliza os dados primeiro), a modelagem no Cassandra é orientada a consultas: primeiro identificam-se as consultas que a aplicação precisa executar, e então se projeta a tabela (table) para atender a essas consultas de forma otimizada.

Princípios fundamentais:

- Uma tabela por padrão de consulta (query pattern).
- Dados são desnormalizados para evitar joins.
- Partition key define como os dados são distribuídos.
- Clustering columns determinam a ordenação dentro de uma partição.

3.3 CQL (Cassandra Query Language)

CQL é a linguagem de consulta do Cassandra, com sintaxe semelhante ao SQL, porém com limitações intencionais para garantir desempenho distribuído.

Criação de keyspace e tabela:

```
CREATE KEYSPACE etec WITH replication = {  
    'class': 'SimpleStrategy', 'replication_factor': 3  
};
```

```
CREATE TABLE alunos (  
    ra UUID PRIMARY KEY,  
    nome TEXT,  
    curso TEXT,  
    semestre INT  
);
```

Inserção de dados:

```
INSERT INTO alunos (ra, nome, curso, semestre)
VALUES (uuid(), 'Carlos', 'DS', 3);
```

3.4 Operações CRUD no Cassandra

SELECT – consulta:

```
SELECT * FROM alunos WHERE curso = 'DS';
SELECT nome, semestre FROM alunos WHERE ra = ?;
```

UPDATE:

```
UPDATE alunos SET semestre = 4 WHERE ra = abc123;
```

DELETE:

```
DELETE FROM alunos WHERE ra = abc123;
```

Importante: no Cassandra, UPDATE e INSERT são essencialmente a mesma operação (UPSERT). Se um registro com a mesma chave primária existir, ele é sobrescrito. Além disso, consultas sem a partition key (ou sem um índice secundário) resultam em varreduras ineficientes e devem ser evitadas.

O Cassandra não suporta joins, subconsultas ou agregações complexas. Para cenários que exigem essas operações, recomenda-se utilizar uma camada de processamento externa (como Apache Spark).

Capítulo 4 – Banco de Dados Chave-Valor (Redis)

4.1 Aplicações típicas (cache, sessão, filas)

Redis (Remote Dictionary Server) é um banco de dados chave-valor em memória, conhecido por sua extrema velocidade e versatilidade. Suas principais aplicações incluem:

- Cache de consultas: reduz a carga no banco principal armazenando resultados frequentes.

```
SET usuario:123 '{"nome":"Ana"}' EX 3600 # expira em 1 hora
```

- Gerenciamento de sessão: armazena dados de sessão de usuários com TTL.
- Filas de mensagens: utilizando listas com LPUSH/RPOP ou o módulo Redis Streams.

```
LPUSH tarefas "processar_pedido_456"  
RPOP tarefas
```

- Contadores e rate limiting: operações atômicas com INCR/DECR.

```
INCR contador:visitas
```

- Publicação/assinatura (pub/sub): comunicação em tempo real.

4.2 Estruturas de dados (strings, lists, sets, hashes)

O Redis suporta diversas estruturas de dados, cada uma com comandos específicos:

Strings (cadeias de texto ou binário):

```
SET chave "valor"  
GET chave  
APPEND chave "sufixo"
```

Lists (listas ordenadas):

```
LPUSH frutas "maçã" "banana"  
LRANGE frutas 0 -1
```

Sets (conjuntos sem repetição):

```
SADD alunos "João" "Maria" "João" # João não é repetido  
SMEMBERS alunos  
SINTER turmaA turmaB # interseção entre conjuntos
```

Hashes (mapas de campos-valor):

```
HSET usuario:1 nome "Carlos" idade 20 curso "DS"  
HGETALL usuario:1  
HINCRBY usuario:1 idade 1
```

4.3 Operações CRUD

Create:

```
SET chave "valor"  
HSET produto:1 nome "Camiseta" preco 49.90
```

Read:

```
GET chave  
HMGET produto:1 nome preco
```

Update:

```
SET chave "novo_valor"  
HSET produto:1 preco 39.90
```

Delete:

```
DEL chave  
HDEL produto:1 preco
```

4.4 Transações não-ACID e persistência

O Redis oferece transações através dos comandos MULTI, EXEC, DISCARD e WATCH. Diferentemente de transações ACID tradicionais, as transações no Redis são executadas de forma otimista: todos os comandos são enfileirados e executados sequencialmente sem interrupção (isolamento), mas não há rollback em caso de erro de execução (falta de atomicidade total).

```
MULTI  
SET chave1 "valor1"  
SET chave2 "valor2"  
EXEC
```

Para persistência, o Redis oferece duas estratégias: RDB (snapshots periódicos do conjunto de dados) e AOF (Append-Only File, que registra cada operação de escrita). É possível combinar ambas para maior segurança.

No modo RDB, o Redis salva o banco em disco em intervalos configuráveis (ex.: a cada 5 minutos se pelo menos 100 chaves foram alteradas). No modo AOF, cada comando de escrita é registrado em um arquivo de log, permitindo reconstrução completa em caso de falha.

Capítulo 5 – Outros Modelos e Arquiteturas

5.1 Banco de dados em grafo (Neo4j)

Bancos de dados em grafo armazenam entidades (nós) e seus relacionamentos (arestas) como cidadãos de primeira classe. O Neo4j é o principal representante dessa categoria, utilizando a linguagem Cypher para consultas.

```
CREATE (a:Aluno {nome: "João", idade: 18})
CREATE (d:Disciplina {nome: "BD II"})
CREATE (a)-[:CURSA]->(d)
```

Consulta:

```
MATCH (a:Aluno)-[:CURSA]->(d:Disciplina {nome: "BD II"})
RETURN a.nome, d.nome
```

Grafos são ideais para cenários como redes sociais, sistemas de recomendação, detecção de fraude e análise de rotas.

5.2 Armazenamento de objetos (S3, MinIO)

O armazenamento de objetos trata dados como objetos imutáveis, cada um com um identificador único (key) e metadados. O Amazon S3 (Simple Storage Service) é o padrão de fato, enquanto o MinIO é uma alternativa open-source compatível com a API S3.

Diferentemente de sistemas de arquivos tradicionais, o armazenamento de objetos é projetado para escalar horizontalmente, suportando bilhões de objetos e petabytes de dados. É amplamente utilizado para backups, data lakes, arquivos multimídia e logs.

5.3 Mecanismos de busca (Elasticsearch)

Elasticsearch é um motor de busca e análise distribuído baseado no Apache Lucene. Ele indexa documentos JSON e oferece busca full-text em tempo real com alta relevância.

```
POST /alunos/_doc
{ "nome": "Maria", "cidade": "São Paulo", "cursos": ["DS", "BD"] }
```

Busca:

```
GET /alunos/_search
{ "query": { "match": { "cidade": "São Paulo" } } }
```

Além de busca textual, o Elasticsearch suporta agregações, análise geoespacial e integração com a stack Elastic (Logstash, Kibana) para observabilidade e análise de logs.

5.4 Streaming de eventos (Kafka)

Apache Kafka é uma plataforma de streaming de eventos distribuída. Ele organiza os eventos em tópicos, que são mantidos por brokers e consumidos por produtores e consumidores.

Kafka é utilizado para pipelines de dados em tempo real, ingestão de logs, processamento de fluxos (Kafka Streams, ksqldb) e integração entre microsserviços. Diferentemente de filas tradicionais, Kafka mantém o histórico de mensagens por um período configurável, permitindo reprocessamento.

5.5 Séries temporais (InfluxDB)

InfluxDB é um banco de dados especializado em séries temporais, otimizado para armazenar e consultar dados indexados por tempo, como métricas de sistemas, leituras de sensores IoT e dados financeiros.

```
INSERT temperatura,local=servidor1,host=web01 valor=38.5 1711400000
SELECT * FROM temperatura WHERE time > now() - 1h
```

Recursos como retenção automática (retention policies), continuous queries e downsampling tornam o InfluxDB ideal para monitoramento e observabilidade.

5.6 Bancos de dados espaciais (PostGIS) e vetores (Qdrant)

PostGIS é uma extensão do PostgreSQL que adiciona suporte a objetos geográficos e operações espaciais, permitindo consultas como distância entre pontos e interseção de polígonos.

```
SELECT nome FROM cidades
WHERE ST_Distance(geom, ST_SetSRID(ST_MakePoint(-46.63, -23.55), 4326)) < 100000
```

Já o Qdrant é um banco de dados vetorial projetado para buscas de similaridade em embeddings gerados por modelos de machine learning (IA generativa, processamento de linguagem natural). Ele permite encontrar itens semanticamente similares com base em distância cosseno ou euclidiana.

```
PUT /collections/produtos
{ "vectors": { "size": 384, "distance": "Cosine" } }
```

A combinação de bancos espaciais e vetoriais está na base de aplicações modernas como geolocalização inteligente, sistemas de recomendação e busca semântica.

Exercícios de Fixação

Capítulo 1 – Fundamentos de Bancos de Dados Não Relacionais

1. Explique a diferença entre NoSQL e NewSQL, citando um exemplo de cada.
2. Descreva o Teorema CAP e dê um exemplo de sistema que prioriza cada combinação (CA, CP, AP).
3. Cite três vantagens e três desvantagens dos bancos NoSQL em relação aos relacionais.
4. O que é consistência eventual? Em que cenários ela é aceitável?
5. Classifique os seguintes dados como estruturados, semiestruturados ou não estruturados: tabela de clientes, arquivo JSON de pedidos, fotos de produtos, feed de rede social.

Capítulo 2 – MongoDB

1. Crie uma coleção chamada 'produtos' e insira três documentos com campos nome, preco e categoria.
2. Escreva uma consulta que retorne todos os produtos com preço maior que 50, ordenados do mais caro para o mais barato.
3. Utilizando o pipeline de agregação, agrupe os produtos por categoria e calcule o preço médio de cada grupo.
4. Explique o funcionamento das transações no MongoDB. Desde qual versão elas estão disponíveis?
5. O que é projeção no MongoDB? Escreva uma consulta que retorne apenas os campos nome e preco dos produtos.

Capítulo 3 – Cassandra

1. Crie um keyspace chamado 'loja' com fator de replicação 3.
2. Explique a diferença entre partition key e clustering columns no Cassandra.
3. Por que no Cassandra não se deve modelar os dados de forma normalizada como no relacional?
4. Escreva uma query CQL para inserir um pedido com ID, cliente e valor total.
5. O que acontece se você executar um UPDATE em um registro que não existe no Cassandra?

Capítulo 4 – Redis

1. Utilizando hashes, armazene os dados de um usuário (nome, email, idade) e recupere todos os campos.
2. Explique como o Redis pode ser usado para implementar um sistema de cache com expiração automática.

3. O que são transações no Redis? Qual a diferença para transações ACID tradicionais?
4. Crie uma lista de tarefas no Redis e simule uma fila (inserir à esquerda, remover à direita).
5. Descreva as diferenças entre persistência RDB e AOF no Redis. Quando usar cada uma?

Capítulo 5 – Outros Modelos e Arquiteturas

1. No Neo4j, crie nós para 'Aluno' e 'Professor' e relacione-os com 'ENSINA' e 'APRENDE'.
2. Qual a diferença entre Amazon S3 e um sistema de arquivos tradicional?
3. Como o Elasticsearch indexa documentos para busca full-text?
4. Explique o papel do Apache Kafka em uma arquitetura de microsserviços.
5. Para que serve um banco de dados vetorial como o Qdrant? Dê dois exemplos de aplicação.

Referências Bibliográficas

- SADALAGE, P. J.; FOWLER, M. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.
- MONIRUZZAMAN, A. B. M.; HOSSAIN, S. A. NoSQL Database: New Era of Databases for Big Data Analytics – Classification, Characteristics and Comparison. International Journal of Database Theory and Application, v. 6, n. 4, 2013.
- CHANDRA, D. G. An Analysis of NoSQL Database. International Journal of Advanced Research in Computer Science and Software Engineering, v. 5, n. 4, 2015.
- HARRISON, G. Next Generation Databases: NoSQL, NewSQL, and Big Data. Apress, 2015.
- MongoDB, Inc. MongoDB Documentation. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 25 jul. 2026.
- Apache Software Foundation. Apache Cassandra Documentation. Disponível em: <https://cassandra.apache.org/doc/>. Acesso em: 25 jul. 2026.
- Redis Ltd. Redis Documentation. Disponível em: <https://redis.io/docs/>. Acesso em: 25 jul. 2026.
- NEO4J, Inc. Neo4j Documentation. Disponível em: <https://neo4j.com/docs/>. Acesso em: 25 jul. 2026.
- ELASTIC. Elasticsearch Documentation. Disponível em: <https://www.elastic.co/guide/>. Acesso em: 25 jul. 2026.
- Apache Software Foundation. Apache Kafka Documentation. Disponível em: <https://kafka.apache.org/documentation/>. Acesso em: 25 jul. 2026.
- InfluxData. InfluxDB Documentation. Disponível em: <https://docs.influxdata.com/>. Acesso em: 25 jul. 2026.
- QDRANT. Qdrant Documentation. Disponível em: <https://qdrant.tech/documentation/>. Acesso em: 25 jul. 2026.
- MinIO, Inc. MinIO Documentation. Disponível em: <https://min.io/docs/>. Acesso em: 25 jul. 2026.
- BREWER, E. CAP Twelve Years Later: How the "Rules" Have Changed. Computer, v. 45, n. 2, p. 23–29, 2012.
- GILBERT, S.; LYNCH, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, v. 33, n. 2, p. 51–59, 2002.